

NAFuzzer: Augmenting Network Protocol Fuzzers via Automatic Template and Seed Generation

Ke Yan

*School of Computer
National University of Defense
Technology
Changsha, China
yanke@nudt.edu.cn*

Bo Yu*

*School of Computer
National University of Defense
Technology
Changsha, China
yubo0615@nudt.edu.cn*

Yong Tang

*School of Computer
National University of Defense
Technology
Changsha, China
ytang@nudt.edu.cn*

Xiangdong Kong

*School of Computer
National University of Defense
Technology
Changsha, China
kongxiangdong@nudt.edu.cn*

Chen Chen

*School of Computer
National University of Defense
Technology
Changsha, China
c_chennudt@nudt.edu.cn*

Jin Lei

*School of Computer
National University of Defense
Technology
Changsha, China
jin13@nudt.edu.cn*

Abstract—Fuzzing is one of the most popular software analysis techniques for discovering vulnerabilities. Different from common terminal software, servers should be interacted with well-formed message that conforms to protocol specifications to exercise more functions. However, without protocol specifications, random bit flipping is unlikely to generate valid messages, making fuzzing struggle to traverse the deeper branches of the program. What's more, manually extracting specifications is typically labor-intensive.

In this paper, we present NAFuzzer, a format-aware fuzzing framework for network protocol software. Based on the key insight that similar message fields being parsed at similar locations in the program, NAFuzzer collects constraints of message handling process using replay-based concolic execution, then runs Loop Mapping algorithm to infer the protocol format, and finally uses the format information to generate seeds and perform fuzzing. We implemented and evaluated NAFuzzer on 6 real-world protocol software (LightFTP, Exim, Live555, Net-SNMP, Dnsmasq, TinyHTTPd). The results show that NAFuzzer can extract the message formats of six protocol software with a field recognition accuracy of close to 90%. In fuzzing, it achieves higher branch coverage (up to 27.3%) and discovers more paths (up to 38.2%) compared to state-of-the-art fuzzers without any manual involvement.

Keywords—network protocol fuzzing, concolic execution, format inference

I. INTRODUCTION

In recent years, numerous security incidents have been caused by network protocol software vulnerabilities. Due to business demands, a large number of web servers are exposed to the public network environment, which is extremely prone

to be attacked by malicious users [1]. The damage caused by protocol software vulnerabilities is enormous. For Instance, the famous EternalBlue vulnerability is a flaw in the Windows SMB service and over 100,000 systems were compromised by this vulnerability [2]. To reduce the impact of protocol software vulnerabilities, it is extremely important to enhance the ability to discover security flaws in protocol software.

Fuzzing is one of the popular techniques for discovering software vulnerabilities. However, fuzzing network protocol software is substantially different from common terminal software, network protocol specification describes the format and sequence of session messages [3]. The former determines whether the message content can be parsed properly, while the latter determines in which order the messages should be sent. Under this constraint, interaction with servers must conform to their specifications so that more functions can be performed. The valid format is the basis for the message being parsed, and its accuracy and completeness affect whether the server can receive the message sequence. Therefore, sufficient message format information is needed to guide the fuzzing process.

Researchers have designed diverse techniques to obtain protocol information in fuzzing. Generation-based fuzzers [1, 4] require manual writing of protocol models to guide fuzz testing, this kind of tool is labor-intensive and relies on protocol specification documents; learning-based fuzzers [5, 6] collect traces and infer the protocol format through data analysis, the accuracy of this type of method highly depends on the amount of data. Coverage-driven greybox fuzzing (CGF) has recently emerged as a solution for protocol testing [7]. AFLNET and StateAFL [8, 9] guide the sending message order by constructing a finite state machine, but both fail to consider

protocol characteristics in their mutation strategies. Many format-aware AFL-type tools rarely support network protocol fuzzing, while their mutation strategies are suitable for protocol message formats [10, 11]. However, the specifications they use also need to be written manually.

Dynamic analysis of binary programs is another type of approach to obtain format information. These techniques use taint analysis or symbolic execution techniques to analyze the message parsing process and extract the offset positions that appear in the message. They can obtain message format information more accurately [12, 13]. However, their computational cost is relatively high.

Based on the above observations, we have summarized the following challenges.

- How to extract message format information from software and a limited number of seeds automatically?
- How to use message format information to guide test case generation?
- How to generate more seeds with the protocol format?

To address the challenges, we propose NAFuzzer, a format-aware CGF framework for network protocol software. Our approach draws on the idea of learning from fuzzing [14, 15], making full use of information from fuzzing itself. NAFuzzer is automated and does not need any prior knowledge of the message fields. NAFuzzer firstly collects constraints of message handling process using replay-based concolic execution, then runs Loop Mapping algorithm to infer the protocol format, and finally uses the format information to generate seeds and perform fuzzing.

We implement NAFuzzer as a framework and evaluate on 6 real-world protocols software from Google benchmark. The experimental evaluation shows that NAFuzzer can extract the message formats of six protocol software with a field recognition accuracy of close to 90% without manual involvement. Moreover, NAFuzzer can improve code coverage by up to about 27.3% and discovers more paths (up to 38.2%) compared to state-of-the-art fuzzers without any manual involvement.

In summary, we make the following contributions:

- We propose a Loop Mapping algorithm to infer the protocol format by single replay and concolic execution techniques. The method extracts message format information from a limited number of seeds and protocol software, and generates templates without any other a priori knowledge or manual involvement.
- We design NAFuzzer as a format-aware CGF framework for network protocol software, to demonstrate our approach.
- We evaluate our framework on 6 real-world software. The experiment results show that our framework can automatically infer the format of the seeds (achieves 90%), achieve a higher branch coverage (up to 27.3%) and discover more paths (up to 38.2%) compared to state-of-the-art fuzzers.

II. RELATED WORK

NAFuzzer is a format-aware CGF framework, adding concolic execution workflow to state-of-the-art fuzzing techniques. We will use this section to present solutions for related techniques and our motivation.

A. Network Protocol Fuzzing

Generation-based fuzzers generate testcases using the protocol model [1, 4]. The model specifies both the message format (e.g., field type, message separator) and the message sequence. Fuzzing different protocols often require writing different models, which is labor-intensive and difficult to cover proprietary protocols. Learning-based fuzzers [5, 6] generate protocol models by automatic inference methods. These techniques infer format from a corpus of network traces, using sequence matching techniques and statistical techniques. Learning-based techniques require less manual involvement, but their accuracy may suffer from the quantity of the corpus.

Coverage-driven greybox fuzzing (CGF) is one of the most popular vulnerability discovery techniques [7]. To test stateful network protocol software, AFLNET and StateAFL design algorithms to identify and construct finite state machines (FSM) for server states [8, 9], which they use to guide sequences of testcases. However, both do not have a mutation strategy applicable to message, which causes the message structure to be damaged during mutation. To better facilitate fuzzing of structured files, many CGF tools support adding lists containing keywords, called dictionaries. However, inserting a word block has limited effect, unless this block is inserted exactly on the boundary of the other two blocks.

B. Concolic Execution

In recent years, with increasing computational power, concolic execution (also known as dynamic symbolic execution) has gradually become popular. Concolic execution was first proposed in EXE [16] and has been improved and optimized by KLEE [17] and S2E [18]. The technique uses symbolic variables to model program input, collects constraints of conditional statements in a trace, then uses a constraint solver to solve the constraints, and finally creates inputs to drive the program executes another path [19]. The use of concolic execution techniques can assist in generating program input passing complex branches. In dealing with branches affected by symbolic variables, such methods often fork to explore both branches, which may lead to path explosion. SymQEMU [20] proposed a compile-based symbolic execution approach. It only collects constraints related to current execution, it does not use the fork mechanism and can effectively avoid these problems. SymQEMU uses qemu emulator, which makes it possible to run over different architectures.

C. Motivation

Before presenting the overview of NAFuzzer, we first use an example to illustrate the limitations of existing tools, which motivate our work.

We take the process of fuzzing RTSP protocol software (Live555) with AFLNET as an example. The format of the RTSP protocol message is shown in TABLE I. The processing

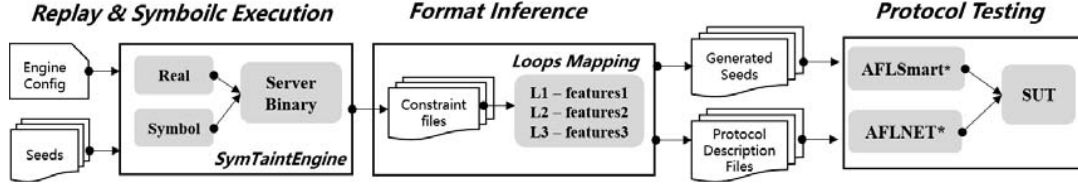


Fig.2. **Overview of NAFuzzer.** It takes the software binary, symbolic execution engine config as input, and runs network protocol fuzzing as a result. Engine configs files are generated from preprocessing stage automatically. The choice of fuzzing engine depends on the characteristics of the target protocol.

method of the message content is determined by the request method field. Using AFL mutation strategy, the format of message can easily be broken. Fig.1 shows the source code that parses the request method of the message.

TABLE I. RTSP MESSAGE FORMAT

Request Method	URL	Version	CRLF
Header Field	Value		CRLF
CRLF			
Content			CRLF

If the request method area “DESCRIBE” is broken, the message cannot properly call “handleCmd_DESCRIBE”. Although this mutation can enter “handleCmd_notSupported” branch, entering the valid data processing branch might require many failed attempts. If the seed contains only some of the methods, it is almost impossible to generate seeds containing other methods by mutation. It is labor-intensive to analyze the software or protocol specification manually.

```

1  } else if (strcmp(cmdName, "DESCRIBE") == 0) {
2      handleCmd_DESCRIBE();
3  } else if (strcmp(cmdName, "SETUP") == 0) {
4      handleCmd_sessionNotFound();
5  } else {
7      handleCmd_notSupported();
8  }

```

Fig. 1. Source Code of the request handling method

III. NAFUZZER OVERVIEW

A. Framework

A core intuition behind the design of NAFuzzer is that message format normally contains header field and content field, the former contains at least one key field to identify the attributes of the latter. The header field is parsed in the shallower branches of the program, while the content field is parsed later in the deeper processing logic.

NAFuzzer makes innovations in the workflow of protocol fuzzing. We apply the concolic execution technique to infer the format of the seeds, and explore the possible values of the header fields based on the format information. In this way, with a limited quantity of seeds, we can extract information about the seed format and generate more seeds, so as to cover more program branches in fuzzing.

Fig.2 provides an overview of our framework, which takes software binaries and seed files as inputs and runs network protocol fuzzing as a result. Firstly, the preprocessing stage analyzes the target software to generate the information required for concolic execution. Then, we replay the seeds a single time and SymTaintEngine collects the constraint information during message handling. After that, the format learning module runs the Loop Mapping method to infer the seed format and generate a format template. It uses the template to explore the possible values of the header field and generates more seeds. Finally, NAFuzzer selects fuzzing engine and starts testing with seeds and format templates.

B. SymTaintEngine

We leveraged SymTaintEngine for NAFuzzer’s concolic execution engine, which is based on SymQEMU [18, 21, 22]. SymQEMU is a user-mode dynamic binary analysis platform that uses compilation-based symbolic execution on qemu. When translating programs to TCG ops, SymQEMU compiles the symbol handling directly into the machine code, rather than translating the target program into a complex intermediate representation and performing symbolic processing. This greatly speeds up its processing and provides compatibility. The performance improvement makes it possible to analyze protocol interactions without worrying about the impact of processing timeouts.

We extend SymQEMU to support network protocol implementation testing. SymTaintEngine can perform concolic execution within the specified address scope (*i.e.*, from symbolic start address to end address) and supports symbolizing multiple types of objects (*e.g.*, registers, memory, and pointers). It is worth mentioning that SymTaintEngine can symbolize the same location multiple times, especially for protocol software with multiple interactions, where the same location can be symbolized in multiple interactions. With the information we provide on network system calls, SymTaintEngine can directly analyze network protocol binaries without source code.

C. Preprocessing & Concolic Execution

We perform concolic execution technique during seeds replay. In network protocol fuzzing, the seed usually contains several messages, we can identify separate messages and replay them in order. The details of identifying the delimiter that divides the message will be mentioned in the next subsection. To leverage concolic execution on network protocol software, we focus only on the message processing process rather than the whole execution. Benefiting from our improvements, we provide symbolic marks and symbolic timing (*i.e.*, execution address ranges) as input to

SymTaintEngine. We can then symbolize the message as a whole without worrying about path explosion.

In order to obtain symbolic marks and timing of the message processing, we summarize the working pattern of network-related system calls and design extraction methods. In this way, NAFuzzer can analyze binary programs without modifying the program. The network-related system calls are as follows: `recv`, `read`, and `recv_msg`.

D. Format Inference

In protocol software fuzzing, random bit flipping can destroy the original format of the message, which causes the exploration to struggle in shallow branches. In format inference stage, we utilize the constraint information to generate message format templates and more test seeds.

1) Loop Mapping Inference Method

Noe Lutz mentions that “Loops are a recurring feature in decryption algorithms” in his paper [23]. Encryption functions tend to process data with the same attributes in a looped mode. The process of parsing messages is similar to the processing of cryptographic functions. In this paper, we design a method called Loop Mapping to infer the message format. We identify loops in the path of a single replay, map the loops with format features, and finally use the constraints associated with the loops to infer format.

In previous work, Xu and Calvet detected loop bodies [24] based on repeated instruction sequences. Our method considers the repeating PC (Program Counter) in trace as a loop body. The choice is less precise than previous work, but it does not require additional information logging work during concolic execution. In moreover, the inference format with this precision is sufficient for guiding fuzzing. Next, we will describe how to identify loop bodies in traces.

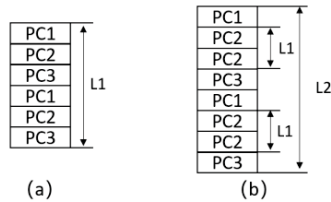


Fig.3. Loop examples in trace

Fig.3 shows two examples of loops according to our definition. In Fig.3 (a), the trace [PC1, PC2, PC3] is repeated at least twice, which meets our criteria for detecting loops. In Fig.3 (b), small loops appear inside large loops, and we call them nested loops. Our loop detection method can detect both types of loop bodies.

The execution trace contains many PCs, and there is a temporal sequential relationship between the appearance of PCs. We abstract the execution trace as a directed graph G and the loop body as a circle, so the loop detection problem is transformed into the problem of finding all circles in a directed graph.

The loop detection algorithm is described as Algorithm 1. We use the adjacency table to describe the directed graph, the

starting point is a symbolic start address. The search process is performed recursively, using a deep first search for successor nodes. Two lists are maintained during the search: the visited list is used to record the nodes that have been visited, and the trace list is used to record the search process. When a node appears in both lists, a new ring is found and will be recorded in the ring list. When the algorithm is finished, all the circles in the directed graph are output.

Algorithm 1: Loop Detection Algorithm - DetectLoop()

Input:
node index of trace graph : $node_i$;
Output:
circle list: C ;

```

1 visited node list: visited; trace list: trace;
2 if  $node_i$  in visited then
3   if  $node_i$  in trace then
4      $trace_i \leftarrow \text{getIndex}(\text{trace})$ 
5      $circle \leftarrow \text{trace}[trace_i, \text{len}(\text{trace})]$ 
6      $C.add(circle)$ 
7     return
8   end
9   return
10 end
11 if  $node_i$  not NULL then
12    $children \leftarrow \text{getChildren}(node_i)$ 
13   foreach child in children do
14     DetectLoop(child)
15     pop(trace)
16   end
17 end
```

After detecting the loops in the execution trajectory, we process the constraints separately based on associated PCs within the loop body. Referring to the reasoning approach of Ployplot [25], our goal is to identify the bounds of the fields in the message format and the constraints that existed in them. We describe the message format based on field and define the following attributes (Example shown in Fig.4).

- Separator: separator is used to specify the boundary of the field.
- Normal field: the part of the message that exists between separators or a set of continuous positions.
- Keyword field: a constant fixed in the format, used for attribute description, often found in text-based protocols.
- Constraint field: stores information of another field location, such as length field.

To identify the separator and normal domain, we analyze the constraint information of the consecutive memory addresses within the loop. Consecutive memory address information is used to identify the boundaries of fields, and the characters compared in constraints are used to identify the separator; To identify keyword domains, we summarize the common string comparison functions (`strcmp`, `strncmp`, `strcasemp`, `strncasemp` and `sscanf`), they use loop comparisons to implement the function. We merge the content of the constraints that are compared with the consecutive addresses in the loop body; as for constraint fields, it is possible to observe such software behavior using concolic execution technique. Taking the length attribute field as an example, the constraint can reflect that the position of the length field will have an offset relationship with its content.

We can identify the length attribute field by analyzing the offset between addresses with their content.

After completing the inference of the above attribute fields, the format inference module combines the location information of fields and outputs message format template. The template is described in XML format for ease of utilization in the next stage.

2) Seed Generation

For most protocols, the protocol message format usually consists of header field and content field. As mentioned in the motivation example in subsection 3.1, the identification of header region attribute fields often plays a role like the magic number, and it is inefficient to explore the branches of string comparison using mutation methods.

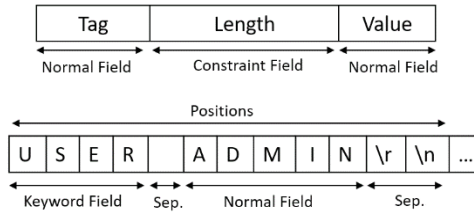


Fig.4. Fields in TLV Structure and FTP Login 'USER ADMIN'

We design a method for header value exploration and seed construction based on message templates. If not specified manually, NAFuzzer explores "interesting" fields sequentially by position: keyword fields in text-based protocols, and fields within a few bytes in binary-based protocols [19]. For an explored field, we use the field's boundary information to filter the constraint information, then extract all possible values compared at the relevant position, and finally use values replacing the same position to construct a new seed. If more than one field is involved, we perform a cross-combination.

```

1  switch (pdu->command) {
2      case SNMP_MSG_TRAP:
3      case SNMP_MSG_RESPONSE:
4      case SNMP_MSG_REPORT:
5      case SNMP_MSG_TRAP2:
6      case SNMP_MSG_INFORM:
7      case SNMP_MSG_GET:
8      case SNMP_MSG_GETNEXT:
9      case SNMP_MSG_GETBULK:
10     case SNMP_MSG_SET:
11     default: ...

```

Fig.5. Net-SNMP switch-case branches

Taking the SNMP protocol header parsing code as an example (Fig.5), there is a header field that defines the type of message. The parsing logic of the field is a switch-case structure with 8 branches. In NAFuzzer, with the help of template information and the collected constraints, we are easily able to extract the possible values of the field and generate new seeds.

E. Format-aware Protocol Fuzzing

When choosing a CGF engine to perform format-aware protocol fuzzing, we have two considerations:

- The technology should be scalable enough so that we can automate the framework.
- Technology can use a protocol template that contains message field boundaries information.

Many format-aware CGF engines do not support fuzzing over network [10, 11], but it is not complicated to extend them to fuzz stateless protocol software. This is enough to validate our idea.

Chunk-based CGF AFLSmart leverages a high-level structural representation of the seed file (Fig.6) to generate new files. AFLSmart adds smart or higher-order strategies such as adding, deleting, and splicing chunks in the virtual structure. Therefore, we believe that AFLSmart meets our requirements: the description files can be automatically generated by the format inference module; The chunk-based mutation strategy can make full use of the chunk-based format template. We extend AFLSmart as AFLSmart* for stateless protocol software fuzzing in NAFuzzer.

```

1  <DataModel name="Chunk">
2      <String name="ckID" length="4" padCharacter=" "/>
3      <Number name="ckSize" size="32">
4          <Relation type="size" of="Data"/>
5      </Number>
6      <Blob name="Data"/>
7      <Padding alignment="16"/>
8  </DataModel>

```

Fig.6. Generic Chunk Model

The approach described above does not support sending messages in sequence, we designed a header field mutation strategy based on stateful protocol fuzzing technique AFLNET. The idea comes from our motivation: to maintain the header area of the message structure and then guide fuzzer to pass branches of handling header attributes. The post-process mechanism of the AFL-like tools, which can call the library for content trimming during mutation, is extensible. We modified the mutation part of AFLNET: automatically generating a format check library based on message templates, which is used to check the message header structure situation.

In summary, with the two designs described above, NAFuzzer can automatically use message format information to guide fuzzing of both stateless and stateful protocol software.

IV. EVALUATION

We evaluate NAFuzzer on real-world network protocol software from a publicly benchmark. The evaluation includes the following criteria: accuracy of format inference, code coverage, number of paths found, and performance. In this section, we aim to answer the following questions in evaluations:

Q1: How accurate is the format information extracted by NAFuzzer?

Q2: Can NAFuzzer improve fuzzing coverage and find more paths using inferred format information?

Q3: Can NAFuzzer improve fuzzing coverage and find more paths using generated seeds?

A. Evaluation Setup

Target Software. We started our evaluation on ProFuzzBench, a public benchmark for network fuzzers [26]. The benchmark contains 13 open-source real-world protocol software. Most of them are stateful protocol software, so we selected 4 targets (LightFTP, Exim, Live555, Dnsmasq) out of 13 protocols and selected 2 popular stateless protocol targets (TinyHTTPd, Net-SNMP), shown as Table II.

Seed Collection. For fuzzing the 4 targets selected from the benchmark, we use the seeds provided by the benchmark; For fuzzing the other 2 targets, we interact with the server according to the manual and collect the generated traffic as seeds.

Evaluation Setup. We conducted our experiments on one server, with Intel(R) Xeon(R) Silver 4216 CPU @ 2.10GHz cores, 28G RAM and 64-bits Ubuntu 16.04 TLS. In fuzzing evaluation, we tested each software for 24 hours and repeated the experiment five times.

TABLE II. EVALUATION TARGETS

Protocol	Software	Protocol	Software
FTP	LightFTP v2.2	DNS	Dnsmasq v2.73
SMTP	Exim-4.89	HTTP	TinyHTTPd-2013
RTSP	Live555-2019	SNMP	Net-SNMP 5.8

B. Qualitative Analysis of Inference

Evaluation Setup. To evaluate the accuracy of the message format identified by NAFuzzer (Q1), we compare the inferred results with the popular protocol analysis tool Wireshark. As shown in Table IV, we used the evaluation criteria of the protocol reverse engineering tool AutoFormat [27]: $|SF|$ represents the number of identified blocks (*i.e.*, there are four blocks in the FTP command in Fig.4); $|Re|$ represents the match ratio(using $|SF|$ of Wireshark as a base); If a field is divided into multiple fields by NAFuzzer, it is defined as an overly fine-grained field, and we use $|SF_o|$ to count; If several fields are identified by NAFuzzer as a single field, it is defined as a coarse-grained field, and we use $|SF_c|$ to count. Restricted by space, our tables present the inferred results for the most complex messages for each protocol.

Table III shows the results of format inference, with the match ratio averaging at 90%. In fact, the accuracy is higher: since our goal is to infer message formats that can assist in fuzz, fields not involved in parsing cannot be inferred (counted as $|SF_o|$). For example, we cannot identify “User-Agent” and “Accept” fields in RTSP. The root cause is that the server does not parse these two fields. Moreover, the overly fine-grained identification counted by $|SF_c|$ is beneficial to fuzzing. For example, the “rtsp://127.0.0.1:8554/wavAudioTest/track1” field of RTSP, NAFuzzer identifies it as keyword field “rtsp://”

and the common field “127.0.0.1:8554/wavAudioTest/track1”. Such identification reflects the true logic of the processing and reduces the number of tests associated with mutating “rtsp://”.

TABLE III. FORMAT ACCURACY OF NAFUZZER

Software	Wireshark	NAFuzzer				
	$ SF $	$ SF $	$ Re $	$ SF_o $	$ SF_c $	avg
Live555	17	15	88%	1	2	90%
Dnsmasq	9	7	78%	0	2	
Net-SNMP	27	26	96%	0	1	
TinyHTTPd	22	17	78%	0	5	
LightFTP	4	4	100%	0	0	
Exim	4	4	100%	0	0	

The efficiency of format inference is shown in Table IV. NAFuzzer shows a good performance in the evaluation of 6 software. Experimental results show that NAFuzzer can automatically and accurately extract the block boundary information from messages (Q1).

TABLE IV. EFFICIENCY OF FORMAT INFERENCE

Protocol	Software	Size	CET	Offline Time
FTP	LightFTP	60.7KB	800ms	$\leq 5s$
SMTP	Exim	926.2KB	500ms	$\leq 5s$
RTSP	Live555	3257.7KB	1s	$\leq 5s$
DNS	Dnsmasq	400.5KB	10min	$\leq 10s$
SNMP	Net-SNMP	14436KB	25min	$\leq 10s$
HTTP	TinyHTTPd	62.9KB	500ms	$\leq 5s$

CET: Concolic execution time per message

C. Code Coverage & Path Found

Experimental setup: To answer questions Q2, Q3, we designed six sets of experiments. The experimental evaluation compares NAFuzzer with two base-line fuzzers:

- AFLNWE: It is an extension of AFL to support network protocol fuzzing, using AFL’s mutation strategy.
- AFLNET: The first network protocol CGF tool for stateful network protocol software, adding a message sequence mutation strategy.

When testing stateless protocol targets, we use AFLNWE as a base-line fuzzer. because only the mutation strategy used by NAFuzzer differs from that of AFLNWE; As for stateful protocol targets, we use AFLNET as a base-line fuzzer because NAFuzzer does extensive work on top of AFLNET. NAFuzzer represents using the same initial seed as the base-line fuzzer, while NAFuzzer-F represents using generated seed. In addition, when testing text-based protocols (LightFTP, Exim, Live555), AFLNET-DICT represents the mode using dictionary, which can reflect the best performance of the base-line fuzzer with manual involvement. Restricted by space, our table represents branch coverage results using line coverage.

To answer Q2, we used NAFuzzer in comparison with the base-line fuzzer. Table V shows the line coverage for each target after 24 hours fuzzing. NAFuzzer clearly outperforms both AFLNWE and AFLNET. The experimental results show that NAFuzzer improves at least 10% on average from base-line fuzzers. For targets with complex message format (Dnsmasq, TinyHTTPd), the gap is larger (10.3%, 27.6%). But

for Exim, NAFuzzer has similar coverage to AFLNWE, probably due to the simpler message format. NAFuzzer has comparable or even lower coverage compared to AFLNET-DICT, because the dictionary can provide more information about the seed content. The experimental results show that we can use message format templates to improve network protocol fuzzing(Q2).

TABLE V. AVERAGE BRANCH COVERAGE

Software	Fuzzer	Line Coverage(%)	Exec Speed	Crashes
Dnsmasq	AFLNWE	12.8	34.03	77
	NAFuzzer	14.1	33.99	96
	NAFuzzer-F	14.2	34.53	101
TinyHTTPd	AFLNWE	52.1	25.47	0
	NAFuzzer	66.3	25.65	0
	NAFuzzer-F	66.3	25.65	0
Net-SNMP*	AFLNWE	10.0	11.39	0
	NAFuzzer	12.7	10.97	0
	NAFuzzer-F	17.3	11.03	0
LightFTP	AFLNET	48.3	5.62	0
	AFLNET-DICT	55.6	5.41	0
	NAFuzzer	52.5	4.46	0
	NAFuzzer-F	59.3	4.32	0
Exim	AFLNET	12.0	4.16	0
	AFLNET-DICT	12.5	4.02	0
	NAFuzzer	12.5	3.92	0
	NAFuzzer-F	12.8	3.78	0
Live555	AFLNET	22.7	8.92	3
	AFLNET-DICT	24.3	9.13	4
	NAFuzzer	24.5	8.59	6
	NAFuzzer-F	25.2	8.14	8

NET-SNMP*: we use map density criterion in AFL for approximate reference
Exec Speed: seed/second

To answer Q3, we used NAFuzzer-F for comparison with NAFuzzer. As shown in Table V, the coverage of NAFuzzer-F and NAFuzzer in the 3 targets (Dnsmasq, TinyHTTPd, Exim) is comparable. The possible reason is that the seeds for these three targets contain most of the protocol methods, and thus we are unable to generate more seeds. However, for 3 targets (SNMP, LightFTP, Live555) the coverage improvement of NAFuzzer-F is relatively obvious because these three protocols contain a rich set of methods. Taking SNMP as an example, NAFuzzer can extract all 8 attribute codes, and it is laborious to collect traffic covering all 8 attribute codes. The experimental results show that we can improve branch coverage and find more paths with the generated seeds (Q3).

By comparing NAFuzzer-F with AFLNET-DICT, the experimental results show that NAFuzzer-F achieves higher coverage in all three text-based protocol software, which also indicates that our method can work better than manual extraction of dictionary information.

As shown in Table V, the line coverage of NAFuzzer is the highest, while the coverage improvement of 3 target is limited. Since in this paper we currently focus on improving the automation of fuzzing, there is still potential for the use of constraint information, such as designing mutation strategies with more constraint information. Moreover, when the number of seeds is limited or there are dependencies within the seed format, the improvement of NAFuzzer will be more obvious.

Finding more paths is one of the foundations to improve coverage. Fig.7 shows the number of paths discovered over time. NAFuzzer-F achieved the highest number of paths found in all six tests. In Dnsmasq, TinyHTTPd and LightFTP, NAFuzzer-F discovers 13.1%, 11.0%, 38.2% more paths than base-line fuzzer. Since the testcases we provide cover all the two methods supported by TinyHTTPd, the inference module cannot generate more seeds, so the lines of NAFuzzer and NAFuzzer-F are overlapping.

In summary, the evaluation shows that NAFuzzer can extract message formats from software and a limited number of seeds, improve coverage and discover more paths with the generated seeds and the format information.

Moreover, the components of NAFuzzer are loosely-coupled to the target, eliminating the need to manually modify the framework for different targets.

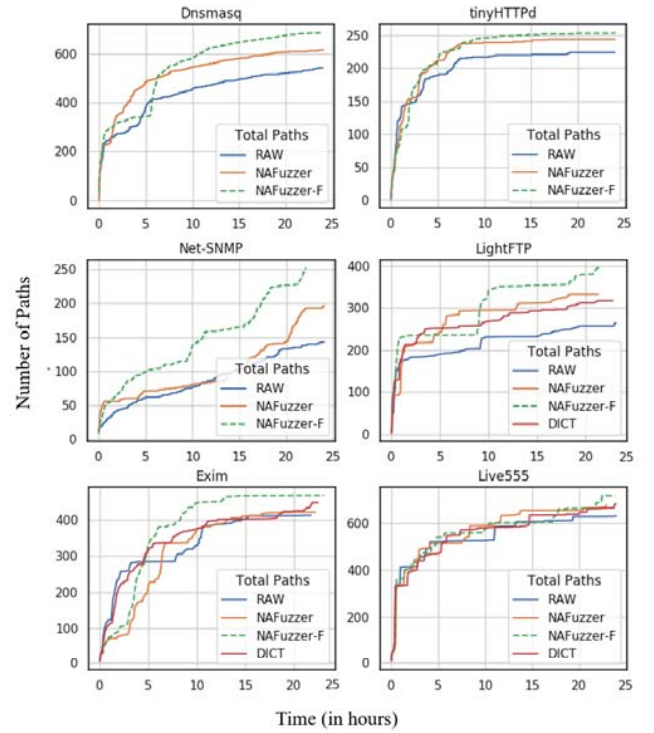


Fig.7. Number of paths discovered over time.

V. DISCUSSION

NAFuzzer uses concolic execution techniques to assist network protocol fuzzing to aware message formats. This allows NAFuzzer to benefit from the advantages of both software analysis techniques. In this section, we will discuss the limitations of NAFuzzer and future research directions to further improve its applicability.

A. Limitation

Compilation Optimization. In concolic execution, one of the inputs to NAFuzzer is the target binary software. In experiments, we observed that the constraints collected by SymTaintEngine are influenced by the way the binary is

compiled. Compilation optimization may merge symbolic branches, reducing the accuracy of format inference.

B. Future Work

Accuracy. The message format inferred by NAFuzzer can be affected by the way the target software is compiled. In the future, we will increase the accuracy of constraint identification, such as adding the analysis of data bit-level operations in comparisons.

Experimental targets. Our experiments have selected six real-world software, and the results show that our approach can automatically extract format information and perform fuzzing. In the future, we will analyze private network protocol implementations, especially for embedded device targets that use a lot of private protocols.

VI. CONCLUSION

In this paper, we present NAFuzzer, a framework augmenting test performance for fuzzing network protocol via automatic template and seed generation. We designed the fuzzers to not rely on manual customization of protocol formats, in order to make protocol fuzzing more efficient. NAFuzzer firstly collects constraints of message handling process using replay-based concolic execution, then runs Loop Mapping algorithm to infer the protocol format, and finally uses the format information to generate seeds and perform fuzzing. We applied NAFuzzer on 6 diverse network software to evaluate its capabilities. The components of NAFuzzer are loosely-coupled to the target, eliminating the need to manually modify the framework for different targets. In format inference process, NAFuzzer achieves 90% domain information identification accuracy, which is comparable to that of protocol analysis tools. The method causes a lower overhead compared to manual analysis. In fuzzing, NAFuzzer achieves a higher branch coverage (up to 27.3%) and discovers more paths (up to 38.2%) compared to state-of-the-art fuzzers.

ACKNOWLEDGMENT

This work was supported by the Natural Science Foundation of China (61902416, 61902412) and Natural Science Foundation of Hunan Province in China (2019JJ50729).

REFERENCES

- [1] Song C, Yu B, Zhou X, and Yang Q. "SPFuzz: a hierarchical scheduling framework for stateful network protocol fuzzing," *IEEE Access* 7 (2019): 18490-18499.
- [2] Chen X. "Cybersecurity Capability Construction: Coordination of Consciousness, Management and Technology—Thoughts Triggered by the "Eternal Blue" Event," *Journal of Information Security Research* 3.8 (2017): 0.
- [3] Y Yu, Z Chen, S Gan, and X Wang. "SGPFuzzer: A state-driven smart graybox protocol fuzzer for network protocol implementations," *IEEE Access* 8 (2020): 198668-198678.
- [4] Pereyda J. boofuzz Documentation[J]. 2019.
- [5] Zhao H, Li Z, Wei H, Shi J, and Huang Y. "SeqFuzzer: An industrial protocol fuzzing framework from a deep learning perspective," 2019 12th IEEE Conference on software testing, validation and verification (ICST). IEEE, 2019.
- [6] H. Gascon, C. Wressnegger, F. Yamaguchi, D. Arp, and K. Rieck. "Pulsar: Stateful black-box fuzzing of proprietary network protocols," *International Conference on Security and Privacy in Communication Systems*. Springer, Cham, 2015.
- [7] Website, "American fuzzy lop (afl) fuzzer," <http://lcamtuf.coredump.cx/afl/technicaldetails.txt>, 2017, accessed: 2017-05-13.
- [8] Pham V T, Böhme M, Roychoudhury A. "AFLNet: a greybox fuzzer for network protocols," 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). IEEE, 2020.
- [9] Natella R. "StateAFL: Greybox Fuzzing for Stateful Network Servers," *arXiv preprint arXiv:2110.06253* (2021).
- [10] Wang J, Chen B, Wei L, and Liu Y. "Superion: Grammar-aware greybox fuzzing," 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019.
- [11] Pham V T, et al. "Smart greybox fuzzing," *IEEE Transactions on Software Engineering* 47.9 (2019): 1980-1997.
- [12] Wen S, Feng C, Meng Q, Zhang B, Wu L, and Tang C. "Testing Network Protocol Binary Software with Selective Symbolic Execution," 2016 12th International Conference on Computational Intelligence and Security (CIS). IEEE, 2016.
- [13] Wen S, Meng Q, Feng C, and Tang C. "A model-guided symbolic execution approach for network protocol implementations and vulnerability detection," *PLoS one* 12.11 (2017): e0188229.
- [14] Yue T, Tang Y, Yu B, Wang P, and Wang E. "Learnaf: Greybox fuzzing with knowledge enhancement," *IEEE Access* 7 (2019): 117029-117043.
- [15] Godefroid P, Peleg H, and Singh R. "Learn&fuzz: Machine learning for input fuzzing," 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2017.
- [16] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler. "EXE: Automatically generating inputs of death," *ACM Transactions on Information and System Security (TISSEC)*, 12(2):10, 2008.
- [17] C. Cadar, D. Dunbar, and D. R. Engler. "EXE: Automatically generating inputs of death," *ACM Transactions on Information and System Security (TISSEC)* 12.2 (2008): 1-38.
- [18] Chipounov V, Kuznetsov V, and Candea G. "S2E: A platform for in-vivo multi-path analysis of software systems," *Acm Sigplan Notices* 46.3 (2011): 265-278.
- [19] Stephens N, Grosen J, and Salls C. "Driller: Augmenting fuzzing through selective symbolic execution," *NDSS*. Vol. 16. No. 2016. 2016.
- [20] Poeplau S, Francillon A. "SymQEMU: Compilation-based symbolic execution for binaries," *NDSS*. 2021.
- [21] S. Poeplau, and A. Francillon. "Symbolic execution with {SymCC}: Don't interpret, compile," 29th USENIX Security Symposium (USENIX Security 20). 2020.
- [22] Yun I, Lee S, Xu M, Jang Y, and Kim, T. "{QSYM}: A practical concolic execution engine tailored for hybrid fuzzing," 27th USENIX Security Symposium (USENIX Security 18). 2018.
- [23] Rhee S Y, Mutwil M. "Towards revealing the functions of all genes in plants," *Trends in plant science* 19.4 (2014): 212-221.
- [24] Xu D, Ming J, and Wu D. "Cryptographic function detection in obfuscated binaries via bit-precise symbolic loop mapping," 2017 IEEE Symposium on Security and Privacy (SP). IEEE, 2017.
- [25] J. Caballero, Yin H, Liang Z, Song D. "Polyglot: Automatic extraction of protocol message format using dynamic binary analysis," *Proceedings of the 14th ACM conference on Computer and communications security*. 2007.
- [26] Natella R, Pham V T. "Profuzzbench: A benchmark for stateful protocol fuzzing," *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 2021.
- [27] Lin Z, Jiang X, Xu D, and Zhang X. "Automatic protocol format reverse engineering through context-aware monitored execution," *NDSS*. Vol. 8. 2008